

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет
Практическая работа

Д32: Проектирование и технический дизайн API

Выполнил:
Мальцев И. И.
К3340

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Проектирование и технический дизайн API для backend-приложения сайта поиска работы.

В рамках работы необходимо описать функциональные и нефункциональные требования, выбрать формат реализации API, спроектировать эндпоинты в формате OpenAPI-схемы, а также описать тела запросов, тела ответов и возможные ошибки API.

Ход работы

В качестве предметной области был выбран сайт для поиска работы. В системе предусмотрены две основные роли пользователей: соискатель и работодатель. Соискатель может регистрироваться, входить в личный кабинет, редактировать профиль, создавать резюме, искать вакансии и отправлять отклики. Работодатель может регистрироваться, управлять профилем компании, создавать вакансии, просматривать отклики и изменять их статус.

Backend-приложение планируется реализовать на языке Go. Для взаимодействия клиента и сервера выбрана архитектура REST API, так как она хорошо подходит для работы с ресурсами, выделенными в ERD-диаграмме: пользователями, профилями, резюме, компаниями, отраслями, вакансиями и откликами.

Функциональные требования

- Пользователь должен иметь возможность зарегистрироваться как соискатель или как работодатель.
- Пользователь должен иметь возможность выполнить вход в систему и получить токены доступа.
- Система должна предоставлять информацию о текущем авторизованном пользователе.
- Соискатель должен иметь возможность просматривать и редактировать свой профиль.
- Соискатель должен иметь возможность создавать, редактировать, удалять и просматривать свои резюме.
- Соискатель должен иметь возможность искать вакансии с фильтрацией по отрасли, зарплате, опыту, типу занятости, городу и поисковой строке.
- Соискатель должен иметь возможность откликнуться на вакансию с выбранным резюме и сопроводительным письмом.

- Работодатель должен иметь возможность просматривать и редактировать свой профиль.
- Работодатель должен иметь возможность редактировать данные своей компании.
- Работодатель должен иметь возможность создавать, просматривать, редактировать и деактивировать вакансии.
- Работодатель должен иметь возможность просматривать отклики на свои вакансии и менять статус отклика.

Нефункциональные требования

- API должно использовать JSON как основной формат передачи данных.
- Доступ к закрытым эндпоинтам должен выполняться с помощью JWT-токена в заголовке Authorization.
- API должно возвращать единый формат ошибок для упрощения обработки на frontend-стороне.
- Список вакансий и откликов должен поддерживать пагинацию через параметры page и limit.
- Backend должен быть реализован на Go с разделением ответственности между слоями Handler, Service, Repository и Database.
- Эндпоинты должны соответствовать REST-подходу и использовать HTTP-методы GET, POST, PUT, PATCH и DELETE.
- Система должна проверять права доступа: соискатель не может управлять вакансиями работодателя, а работодатель не может создавать резюме соискателя.

Выбранная архитектура API

Для реализации выбран REST API. Основная идея REST API заключается в том, что каждая сущность системы представляется отдельным ресурсом, а действия над ресурсами выполняются стандартными HTTP-методами.

В проекте используется базовый адрес API: `http://localhost:8080/api/v1`. Авторизация реализуется через Bearer JWT. Открытыми остаются эндпоинты регистрации, входа, просмотра списка отраслей, просмотра компаний и поиска вакансий. Остальные операции требуют авторизации.

Архитектура backend-приложения на Go

Для backend-приложения используется слоистая архитектура:

- Handler - принимает HTTP-запросы, читает параметры и тело запроса, возвращает HTTP-ответ.
- Service - содержит бизнес-логику: проверку ролей, прав доступа, статусов и корректности операций.
- Repository - отвечает за SQL-запросы и взаимодействие с базой данных.
- Database - хранит данные в таблицах, спроектированных в ERD-диаграмме.

Такая структура позволяет отдельно развивать логику API, бизнес-правила и работу с базой данных.

Основные эндпоинты API

Метод	Эндпоинт	Назначение	Раздел
POST	/auth/register	Register user	Auth
POST	/auth/login	Login	Auth
POST	/auth/refresh	Refresh tokens	Auth
GET	/auth/me	Current user	Auth
GET	/industries	List industries	Reference
GET	/companies/{companyId}	Company by id	Reference
GET	/vacancies	Search vacancies	Vacancies
GET	/vacancies/{vacancyId}	Vacancy details	Vacancies
POST	/vacancies/{vacancyId}/applications	Apply to vacancy (applicant)	Vacancies
GET	/applicant/profile	Get applicant profile	Applicant
PUT	/applicant/profile	Update applicant profile	Applicant
GET	/applicant/resumes	List own resumes	Applicant
POST	/applicant/resumes	Create resume	Applicant
PUT	/applicant/resumes/{resumeId}	Update resume	Applicant
DELETE	/applicant/resumes/{resumeId}	Delete resume	Applicant
GET	/applicant/applications	List own applications	Applicant
GET	/employer/profile	Get employer profile	Employer
PUT	/employer/profile	Update employer profile	Employer
GET	/employer/company	Get own company	Employer
PUT	/employer/company	Update company	Employer
GET	/employer/vacancies	List own vacancies	Employer
POST	/employer/vacancies	Create vacancy	Employer
GET	/employer/vacancies/{vacancyId}	Get own vacancy by id	Employer
PUT	/employer/vacancies/{vacancyId}	Update own vacancy	Employer
DELETE	/employer/vacancies/{vacancyId}	Deactivate vacancy	Employer
GET	/employer/vacancies/{vacancyId}/applications	List applications for own vacancy	Employer
PATCH	/employer/applications/{applicationId}/status	Update application status	Employer

Описание тел запросов и ответов

В OpenAPI-схеме были описаны основные DTO-модели, которые будут использоваться в Go-коде как структуры для обработки входящих и исходящих данных.

Регистрация соискателя

POST /auth/register

Тело запроса:

```
{
  "role": "applicant",
  "email": "user@example.com",
  "password": "password123",
  "first_name": "Илья",
  "last_name": "Мальцев",
  "phone": "+79990000000",
  "city": "Санкт-Петербург",
  "about": "Начинающий backend-разработчик"
}
```

Тело ответа:

```
{
  "access_token": "jwt-access-token",
  "refresh_token": "jwt-refresh-token",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "role": "applicant"
  }
}
```

Вход пользователя

POST /auth/login

Тело запроса:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Тело ответа:

```
{
  "access_token": "jwt-access-token",
  "refresh_token": "jwt-refresh-token",
  "user": {
    "id": "uuid",
    "email": "user@example.com",
    "role": "applicant"
  }
}
```

Создание резюме

POST /applicant/resumes

Тело запроса:

```
{
  "title": "Junior Backend Developer",
  "experience_years": 1,
  "skills": "Go, SQL, REST API, Git",
  "education": "Университет ИТМО",
  "work_experience": "Учебные проекты на Go",
  "expected_salary": 80000,
  "is_active": true
}
```

Тело ответа:

```
{
  "id": "uuid",
  "applicant_id": "uuid",
  "title": "Junior Backend Developer",
  "experience_years": 1,
  "skills": "Go, SQL, REST API, Git",
  "education": "Университет ИТМО",
  "work_experience": "Учебные проекты на Go",
  "expected_salary": 80000,
  "is_active": true
}
```

Поиск вакансий

GET /vacancies?

industry_id=uuid&salary_from=50000&experience_level=junior&page=1&limit=20

Тело запроса:

Тело запроса отсутствует, фильтры передаются через query-параметры.

Тело ответа:

```
{
  "data": [
    {
      "id": "uuid",
      "title": "Junior Go Developer",
      "company_id": "uuid",
      "employer_id": "uuid",
      "industry_id": "uuid",
      "description": "Разработка backend-сервисов",
      "requirements": "Go, SQL, REST API",
      "salary_from": 70000,
      "salary_to": 120000,
      "experience_level": "junior",
      "employment_type": "full_time",
      "city": "Санкт-Петербург",
      "is_active": true
    }
  ],
  "meta": {
    "page": 1,
    "limit": 20,
    "total": 1
  }
}
```

Создание вакансии работодателем

POST /employer/vacancies

Тело запроса:

```
{
  "industry_id": "uuid",
  "title": "Junior Go Developer",
  "description": "Разработка backend-сервисов",
  "requirements": "Go, SQL, REST API",
}
```

```
"salary_from": 70000,
"salary_to": 120000,
"experience_level": "junior",
"employment_type": "full_time",
"city": "Санкт-Петербург",
"is_active": true
}
```

Тело ответа:

```
{
  "id": "uuid",
  "company_id": "uuid",
  "employer_id": "uuid",
  "industry_id": "uuid",
  "title": "Junior Go Developer",
  "description": "Разработка backend-сервисов",
  "requirements": "Go, SQL, REST API",
  "salary_from": 70000,
  "salary_to": 120000,
  "experience_level": "junior",
  "employment_type": "full_time",
  "city": "Санкт-Петербург",
  "is_active": true
}
```

Отклик на вакансию

POST /vacancies/{vacancyId}/applications

Тело запроса:

```
{
  "resume_id": "uuid",
  "cover_letter": "Здравствуйте! Хочу откликнуться на вакансию."
}
```

Тело ответа:

```
{
  "id": "uuid",
  "vacancy_id": "uuid",
  "applicant_id": "uuid",
  "resume_id": "uuid",
  "status": "submitted",
  "cover_letter": "Здравствуйте! Хочу откликнуться на вакансию."
}
```

Изменение статуса отклика

PATCH /employer/applications/{applicationId}/status

Тело запроса:

```
{
  "status": "invited"
}
```

Тело ответа:

```
{
  "id": "uuid",
  "vacancy_id": "uuid",
  "applicant_id": "uuid",
  "resume_id": "uuid",
}
```

```

    "status": "invited",
    "cover_letter": "Здравствуйте! Хочу откликнуться на вакансию."
  }

```

Возможные ошибки API

В API предусмотрен единый формат ошибки. Он позволяет frontend-приложению одинаково обрабатывать ошибки валидации, авторизации, доступа и отсутствия данных.

```

{
  "error": {
    "code": "BAD_REQUEST",
    "message": "Некорректные данные запроса",
    "details": [
      {
        "field": "email",
        "issue": "Некорректный формат email"
      }
    ]
  }
}

```

Код	Название	Когда возникает
400	Bad Request	Переданы некорректные данные или нарушена валидация.
401	Unauthorized	Пользователь не авторизован или токен недействителен.
403	Forbidden	Пользователь не имеет прав для выполнения операции.
404	Not Found	Запрашиваемый ресурс не найден.
409	Conflict	Конфликт данных, например повторная регистрация email или повторный отклик.

Связь API с ERD-диаграммой

OpenAPI-схема была спроектирована на основе ERD-диаграммы базы данных из первой работы. Таблица users соответствует эндпоинтам авторизации, applicant_profiles и resumes используются в разделе Applicant, employer_profiles и companies используются в разделе Employer, industries применяется как справочник для фильтрации, vacancies отвечает за поиск и управление вакансиями, а applications хранит отклики соискателей на вакансии.

Вывод

В результате выполнения практической работы был выполнен технический дизайн REST API для backend-приложения сайта поиска работы. Были описаны функциональные и нефункциональные требования, выбран формат REST API, определена архитектура backend-приложения на Go и спроектированы основные эндпоинты в формате OpenAPI.

Разработанная OpenAPI-схема покрывает основные сценарии системы: регистрацию и вход пользователей, работу с профилями соискателей и работодателей, создание и поиск вакансий, управление резюме, отправку откликов и изменение статусов откликов. Полученная спецификация может быть использована для генерации Swagger-документации и дальнейшей реализации backend-приложения на Go.